Take-Home Assignment: Geriatric Care Assessment Form

Role: Frontend Developer **Stack:** React 19 + TypeScript + Mantine + Zod **Time budget:** 2 hours

Scoped to fit that window. If you pass three hours, stop, commit what works, and say in your README what you would do next.

---

1. The task

Build a one-page **Geriatric Care Assessment form**: 10 fields a visiting nurse fills in when checking on an elderly patient at home.

Start from Mantine's Vite template, build the form with @mantine/form, and validate it with the Zod schema in section 3. **The schema is given to you.** Your job is the UI and the wiring, not inventing the rules.

Use made-up data only. No real patient names, record numbers, or phone numbers anywhere in your repo.

Links

- Template: https://github.com/mantinedev/vite-template
- Mantine form: https://mantine.dev/form/package/
- Schema validation with schemaResolver: https://mantine.dev/form/schema-validation/
- Zod: https://zod.dev/
- Mantine components: https://mantine.dev/core/package/ and https://mantine.dev/dates/date-input/

2. Setup

```
npx degit mantinedev/vite-template geriatric-assessment-form
cd geriatric-assessment-form
git init && git add -A && git commit -m "chore: mantine vite template"

yarn install                                  # template uses Yarn 4
yarn add @mantine/form @mantine/dates dayjs zod
yarn dev
```

Run yarn test now, before you write anything. It runs typecheck, lint, format check, tests, and build, and it must still pass when you submit. Finding out about formatting rules at the end is a bad way to lose half an hour.

3. The schema

Create `src/features/assessment/schema.ts` with exactly this. Reformat if you like, but do not change the rules or the messages.

```
import { z } from 'zod';

export const MOBILITY = ['independent', 'cane', 'walker', 'wheelchair', 'bedbound'] as
const;

// '2026-08-07' -> '1966-08-07'. ISO date strings compare correctly with < and >.
const minus60Years = (iso: string) => `${Number(iso.slice(0, 4)) -
60}${iso.slice(4)}`;

export const assessmentSchema = z
  .object({
    mrn: z.string().trim().regex(/^MRN-\d{6}$/, { error: 'Must look like MRN-004821'
}),

    patientName: z.string().trim()
      .min(2, { error: 'Patient name must be at least 2 characters' })
      .max(60, { error: 'Patient name cannot exceed 60 characters' }),

    dateOfBirth: z.iso.date({ error: 'Date of birth is required' }),

    assessmentDate: z.iso.date({ error: 'Assessment date is required' }),

    mobility: z.enum(MOBILITY, { error: 'Select a mobility status' }),

    barthelIndex: z.number({ error: 'Barthel Index score is required' })
      .int({ error: 'Barthel Index must be a whole number' })
      .min(0, { error: 'Barthel Index ranges from 0 to 100' })
      .max(100, { error: 'Barthel Index ranges from 0 to 100' })
      .multipleOf(5, { error: 'Barthel Index is scored in steps of 5' }),

    medicationCount: z.number({ error: 'Enter the number of regular medications' })
      .int({ error: 'Medication count must be a whole number' })
      .min(0, { error: 'Medication count cannot be negative' })
      .max(30, { error: 'Enter 30 or fewer' }),

    pharmacistReviewRequested: z.boolean(),
```

```
    followUpDate: z.iso.date({ error: 'Next review date is required' }),

    consentObtained: z.literal(true, {
      error: 'Consent must be obtained before the assessment can be saved',
    }),
  })
  .refine(
    (d) => !d.dateOfBirth || !d.assessmentDate || d.dateOfBirth <=
minus60Years(d.assessmentDate),
    { error: 'This pathway is for patients aged 60 and over', path: ['dateOfBirth'] }
  )
  .refine((d) => !d.followUpDate || !d.assessmentDate || d.followUpDate >
d.assessmentDate, {
    error: 'Next review must be after the assessment date',
    path: ['followUpDate'],
  })
  .refine((d) => d.medicationCount < 5 || d.pharmacistReviewRequested === true, {
    error: 'Five or more medications is polypharmacy: a pharmacist review is
required',
    path: ['pharmacistReviewRequested'],
  });

export type Assessment = z.infer<typeof assessmentSchema>;
```

What the odd bits mean

- **Barthel Index** measures how independently a patient manages daily activities like eating, bathing, and stairs. Ten items scored in steps of 5, so the total is always a multiple of 5. Higher means more independent.
- **Polypharmacy** is five or more regular medicines. It raises the risk of drug interactions, so crossing that count forces a pharmacist review.
- **Age is checked against the assessment date, not today**, because a backdated visit must be judged on the patient's age at the visit.
- This is **Zod 4**: { error: '...' } not { message: '...' }, and z.iso.date() not z.string().date(). Older examples online will not compile.
- z.iso.date() wants a 'YYYY-MM-DD' **string**, which is exactly what Mantine's DateInput gives you. No Date objects anywhere.
- The three .refine() calls are the cross-field rules. Each is guarded so a blank date shows only "required" instead of a stack of follow-on errors.
- Get your value type from Assessment. Do not hand-write a second interface. Empty initial values will not satisfy Assessment, so you will need to type initial values as the input shape and let the resolver produce the parsed output. How you handle that gap is part of what we are looking at.

4. Build this

Field	Label	Component
mrn	Medical record number	TextInput, placeholder MRN-004821
patientName	Patient name	TextInput
dateOfBirth	Date of birth	DateInput
assessmentDate	Assessment date	DateInput, maxDate = today
mobility	Mobility	Select, options from MOBILITY
barthelIndex	Barthel Index	NumberInput, step={5} min={0} max={100}
medicationCount	Regular medications	NumberInput, min={0} max={30}
pharmacistReviewRequested	Pharmacist review requested	Checkbox
followUpDate	Next review date	DateInput
consentObtained	Consent	Checkbox, "Patient or representative has given consent"

Requirements, most important first:

1. All 10 fields render, inside a Paper in a Container, with a heading.
2. **Validation goes through schemaResolver** off the schema. No hand-written validate object. No rule re-implemented in JSX. If you are typing a regex or a >= 5 in a component file, stop.
3. Errors show under each field using Mantine's built-in error rendering. No alert box, no summary at the top.
4. Validation runs on blur and on submit. Untouched fields stay quiet.
5. On a valid submit: wait ~800ms to fake a save, disable the button and show its loading state, then show a success Alert with the parsed values in a Code block. Print what Zod returned, not the raw form state.
6. A **"Load sample patient"** button fills the form with the valid fixture below.

Build the Select options from the MOBILITY array, turning home_visit-style values into readable labels. Adding a value to the array should make it appear in the dropdown with no other edit.

Out of scope: no backend, no auth, no routing, no state library, no toast notifications, no Storybook, no custom styling, no responsive work beyond what Mantine gives you for free.

5. Fixtures

Valid, and all invented:

```
{
  mrn: 'MRN-004821',
  patientName: 'Sushila Deshpande',
  dateOfBirth: '1949-03-12',
  assessmentDate: '2026-08-07',
  mobility: 'cane',
  barthelIndex: 80,
  medicationCount: 3,
  pharmacistReviewRequested: false,
  followUpDate: '2026-09-04',
  consentObtained: true,
}
```

Each single change below must be **rejected**, with the error on the field named:

Change	Error on
mrn: 'MRN-4821'	mrn
patientName: 'S'	patientName
dateOfBirth: '1966-08-08' (one day short of 60)	dateOfBirth
mobility: 'crutches'	mobility
barthelIndex: 82 (not a multiple of 5)	barthelIndex
barthelIndex: 105	barthelIndex

medicationCount: 5 with review unchecked	pharmacistReviewRequested
followUpDate: '2026-08-07' (same day)	followUpDate
consentObtained: false	consentObtained

And these must be **accepted**:

Change	Why
dateOfBirth: '1966-08-07'	Turns 60 on the assessment date
barthelIndex: 0 and barthelIndex: 100	Both ends of the scale are real scores
medicationCount: 4 with review unchecked	Below the polypharmacy threshold

An empty form should give exactly **9** errors, one per required field, with no cross-field noise. Stacked messages on one blank field means something is wrong in your wiring.

6. Tests

Two tests. Vitest and React Testing Library are already set up.

1. **Schema, via `safeParse`:** date of birth exactly 60 versus one day short.
2. **Form, rendered:** load the sample patient, submit, and assert your save handler was called with the parsed values.

`yarn vitest` must pass. We care more about whether you picked a real boundary than about coverage.

7. Submitting

A public Git repo with the working app, plus a README.md covering how to run it, anything unfinished and why, and roughly how long you spent. Be honest about the time; it does not count against you. Show your commit history rather than one big "initial commit". Also, deploy this application on any platform of your choice and mention the URL in the README.md.

Reply to this email with the link.

On AI tools: use them, we do. But you own every line, and we will ask about specific choices on the follow-up call. If a tool wrote something you do not understand, understand it or replace it.

8. How we score this

Area	Weight
It works: 10 fields render, validation fires, valid submit succeeds, no console errors	40%
Schema wiring: schemaResolver used properly, no duplicated rules, types from z.infer	30%
Code quality: sensible files, readable names, no dead code, yarn test green	20%
Tests: two tests that hit a real boundary	10%

Red flags:

- Validation rules copy-pasted into component files.
- A hand-written interface duplicating Assessment.
- any used to silence the resolver's types.
- Errors thrown away for a generic "Invalid input".
- A clinical score silently rounded, clamped, or defaulted instead of rejected. Never quietly change a number a nurse typed.
- Real-looking patient data in the repo.
- Lint or format config deleted to make yarn test pass.

9. If something is unclear

Make a reasonable call, note it in your README, and keep going. Spotting ambiguity and deciding is a signal we want to see. If you are properly stuck, email us.